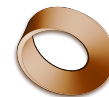


Name: \_\_\_\_\_

Class: \_\_\_\_\_

Mr. Rawson • GCSE Computer Science

Year 10/11 2026



# REFERENCE — Pseudocode, ERL and Python

*Three words that get mixed up constantly. Only one of them is a language you can run.*

## The three things, and how they differ

**1. Pseudocode is a way of writing, not a language.** It is code-shaped English. There is no pseudocode compiler, no official grammar, and no single correct version — two people can write the same algorithm differently and both be right. It exists so you can think about the *steps* without arguing with a machine about semicolons.

**2. ERL is OCR's own pseudocode, written down properly.** ERL stands for **Exam Reference Language**. OCR took the loose idea of pseudocode and published an exact version of it — keywords, operators, the lot — so that every exam question can be printed in one consistent notation.

**Why ERL exists at all.** Schools teach different languages — this one teaches Python, others teach Java, C# or Visual Basic. If OCR printed exam questions in Python, the Python schools would have an unfair head start. So OCR invented a neutral notation that belongs to nobody, and prints every question in that.

**Nothing runs ERL.** There is no ERL interpreter anywhere in the world. It exists only on paper. That is not a flaw — it is the entire point.

**3. Python is a real language.** One fixed grammar, enforced by a program that refuses to run your code if you get it wrong. That refusal is annoying, and it is also the thing that teaches you.

So: **pseudocode is a style, ERL is a written-down dialect of that style, and Python is a language a computer obeys.**

## The good news: ERL looks a lot like Python

### ERL — what the exam prints

```
name = input("Enter a name")
age  = int(input("Enter age"))
const VAT = 0.2

if age >= 17 then
    print("Old enough")
else
    print("Not old enough")
endif
```

### Python — what you type

```
name = input("Enter a name")
age  = int(input("Enter age"))
VAT = 0.2

if age >= 17:
    print("Old enough")
else:
    print("Not old enough")
```

`input()`, `print()`, `=` to assign, `==` to compare, `int()`, `str()`, `float()`, `bool()` — all identical. If you can read Python, you can already read most ERL.

## The four places people actually stumble

Everything else is small. These four are worth learning deliberately.

**1. ERL closes its blocks with a word. Python closes them with indentation.** `endif · endwhile · next i · endfunction · endprocedure · endswitch`. Where Python relies on how far the line is pushed across, ERL says so out loud. *On paper this is an advantage* — indentation is easy to lose when you are writing by hand in a hurry.

**2. Counting loops are written differently.**

ERL — what the exam prints

```
for i = 1 to 10
  print(i)
next i
```

Python — what you type

```
for i in range(1, 11):
  print(i)
```

**3. String handling uses different words.**

Every example below uses the same string — the one OCR uses in the specification itself. Note there is *no space* in it, so it is 15 characters long.

```
subject = "ComputerScience"
```

Counting starts at 0, so the characters sit at these positions:

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| C | o | m | p | u | t | e | r | S | c | i  | e  | n  | c  | e  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

| ERL                                 | Gives             | Python                       |
|-------------------------------------|-------------------|------------------------------|
| <code>subject.length</code>         | 15                | <code>len(subject)</code>    |
| <code>subject.upper</code>          | "COMPUTERSCIENCE" | <code>subject.upper()</code> |
| <code>subject.lower</code>          | "computerscience" | <code>subject.lower()</code> |
| <code>subject.left(4)</code>        | "Comp"            | <code>subject[:4]</code>     |
| <code>subject.right(3)</code>       | "nce"             | <code>subject[-3:]</code>    |
| <code>subject.substring(3,5)</code> | "puter"           | <code>subject[3:8]</code>    |
| <code>ASC('A')</code>               | 65                | <code>ord('A')</code>        |
| <code>CHR(97)</code>                | 'a'               | <code>chr(97)</code>         |

### Two traps in that table.

**1. .length, .upper and .lower have no brackets in ERL.** It is `subject.length`, never `subject.length()`. Python is the other way round — `subject.upper()` needs its brackets. The habit does not transfer, in either direction.

**2. substring(x, i) is *start* and *how many* — not *start and end*.** `subject.substring(3,5)` means *begin at index 3 and take 5 characters*, which gives "puter". Read as a Python slice it would wrongly give "pu". This is the single most common ERL mistake among students who are fluent in Python.

And joining strings: **+ is the only way ERL has.** No f-strings, no `.format()`, no `join()` — so `print("Hello " + name)` is the construction to know.

**4. ERL has a switch statement. Python (mostly) does not** — you would write a chain of `if / elif / else` instead.

And a handful of operator spellings: `MOD` for `%`, `DIV` for `//`, `^` for `**`, `AND/OR/NOT` in capitals, and `//` to start a comment where Python uses `#`.

### What this means in the exam

Paper 2 has two sections, and they treat this differently.

| Where              | What you may answer in                                                  |
|--------------------|-------------------------------------------------------------------------|
| Section A          | Pseudocode, flowcharts, bullet points, ERL, or plain English. All fine. |
| Section B — Design | Pseudocode, flowcharts, tick boxes, plain English. Fine.                |
| Section B — Write  | <b>ERL or a real programming language. Nothing else.</b>                |

**The rule that costs marks.** On a Section B *Write* question, an answer given in loose pseudocode, plain English or bullet points scores **zero** — however good the thinking behind it is. It must be ERL or real code.

**The rule that saves them.** On those same questions, you are **not** penalised for minor syntax errors. A missing bracket or colon will not cost you. The examiners know an IDE would have caught it.

*So: be precise about the logic, and do not panic about the punctuation.*

*You never have to **write** ERL — correct Python is accepted everywhere ERL is.  
But you do have to **read** it, because that is what the questions are printed in.*

The Exam Reference Language is defined in the OCR GCSE (9–1) Computer Science J277 specification, §3c, which OCR publishes free at [ocr.org.uk](http://ocr.org.uk). The examples here are short illustrations written for this class, not a reproduction of that appendix — go to the specification itself for the complete definition.